23CSE111 OBJECT ORIENTED PROGRAMMING

S2 B.Tech CSE

Department of Computer Science and Engineering

Amrita School of Computing, Amritapuri Campus

CASE STUDY

PHASE – 2 (Design)

Architectural Modelling (CO2)

Group No : C3

Team Members :

Roll No	Student Name
AM.SC.U4CSE25225	K V. Pavithra
AM.SC.U4CSE25250	Chathurya Singuru
AM.SC.U4CSE25257	Harshini Vedulla
AM.SC.U4CSE25268	Sriha Gara

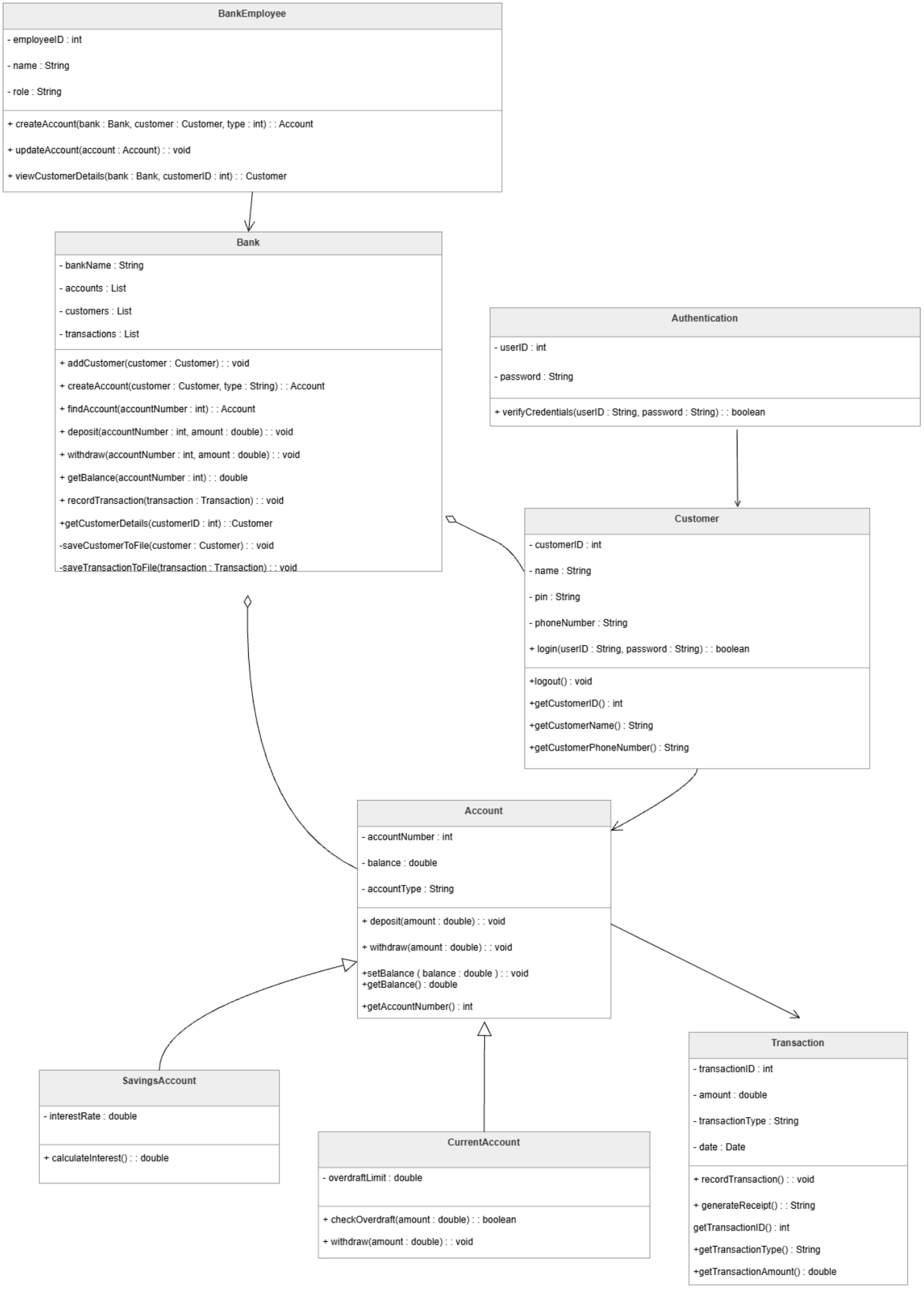
1. Problem Title : Bank Management System [BMS]

STRUCTURAL DESIGN

2. Class diagram (draw class diagram that shows different classes and their relationships. Use UML notations)

The class diagram of the Bank Management System shows the main classes and their relationships. The Bank class manages customers, accounts, and transactions using aggregation. The Customer is linked to the Account, meaning a customer can have multiple accounts. The Account class has inheritance, where SavingsAccount and CurrentAccount are its types. The BankEmployee manages accounts and customer details, and Authentication is used for secure login.

- The system uses file handling for data persistence.
- The Bank class stores customer and transaction details in files,
- Authentication is used internally in the Login use case to verify user credentials.


BankEmployee
- employeeID : int
- name : String
- role : String
+ createAccount(bank : Bank, customer : Customer, type : int) : : Account
+ updateAccount(account : Account) : : void
+ viewCustomerDetails(bank : Bank, customerID : int) : : Customer
Bank
- bankName : String
- accounts : List
- customers : List
- transactions : List
+ addCustomer(customer : Customer) : : void
+ createAccount(customer : Customer, type : String) : : Account
+ findAccount(accountNumber : int) : : Account
+ deposit(accountNumber : int, amount : double) : : void
+ withdraw(accountNumber : int, amount : double) : : void
+ getBalance(accountNumber : int) : : double
+ recordTransaction(transaction : Transaction) : : void
+getCustomerDetails(customerID : int) : :Customer
-saveCustomerToFile(customer : Customer) : : void
-saveTransactionToFile(transaction : Transaction) : : void
Authentication
- userID : int
- password : String
+ verifyCredentials(userID : String, password : String) : : boolean
Customer
- customerID : int
- name : String
- pin : String
- phoneNumber : String
+ login(userID : String, password : String) : : boolean
+logout() : void
+getCustomerID() : int
+getCustomerName() : String
+getCustomerPhoneNumber() : String
Account
- accountNumber : int
- balance : double
- accountType : String
+ deposit(amount : double) : : void
+ withdraw(amount : double) : : void
+setBalance (balance : double) : : void
+getBalance() : double
+getAccountNumber() : int
SavingsAccount
- interestRate : double
+ calculateInterest() : : double
CurrentAccount
- overdraftLimit : double
+ checkOverdraft(amount : double) : : boolean
+ withdraw(amount : double) : : void
Transaction
- transactionID : int
- amount : double
- transactionType : String
- date : Date
+ recordTransaction() : : void
+ generateReceipt() : : String
getTransactionID() : int
+getTransactionType() : String
+getTransactionAmount() : double

BEHAVIUORAL DESIGN

3. Use-case diagram

a. List of Actors

1. Customer – Performs operations like login, deposit, withdraw, and check balance.

2. BankEmployee – Manages accounts and customer details

b. List of Use Cases (with description)

1. Login – The customer enters user ID and pin to access the system. Authentication verifies the credentials.

2. Deposit Money – The customer deposits money into the account, and the system updates the balance and records the transaction.

3. Withdraw Money – The customer withdraws money from the account. The system checks the balance before processing.

4. Check Balance – The customer views the current account balance.

5. Logout – The customer exits the system securely.

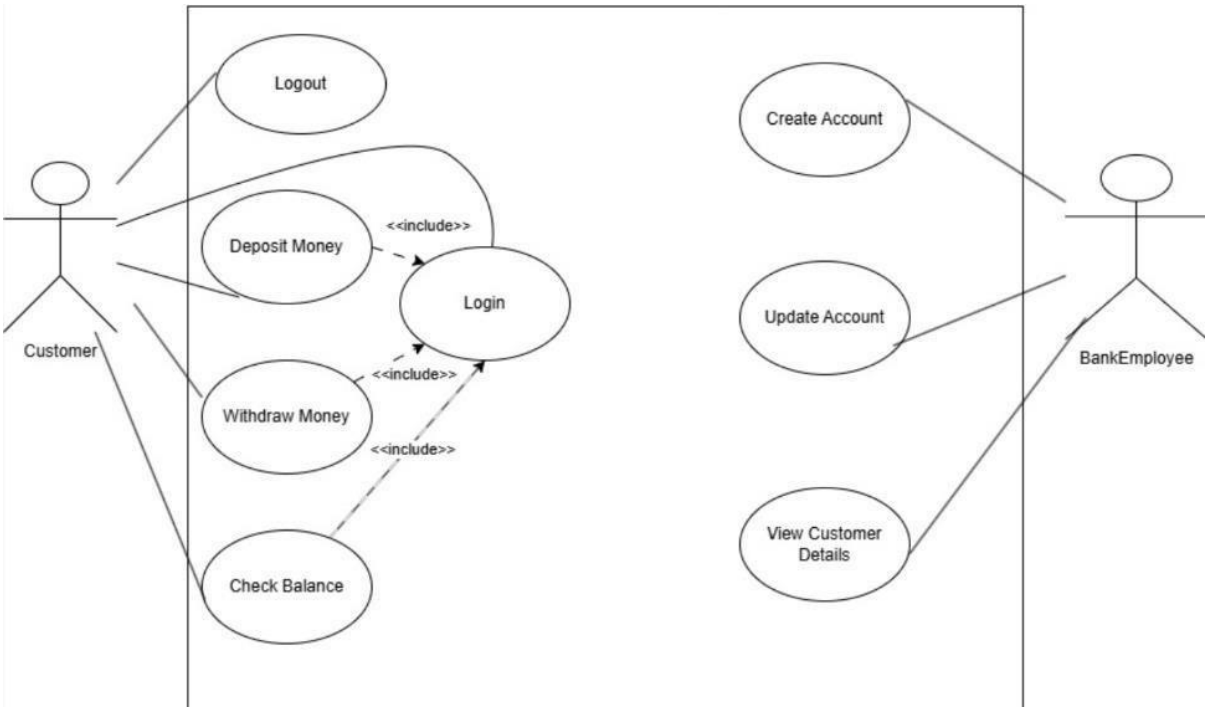
6. Create Account – The bank employee creates a new account for a customer.

7. Update Account – The bank employee updates account details.

8. View Customer Details – The bank employee views customer information.

c. Use case diagram (use UML notations)

Note : All operations require successful login.

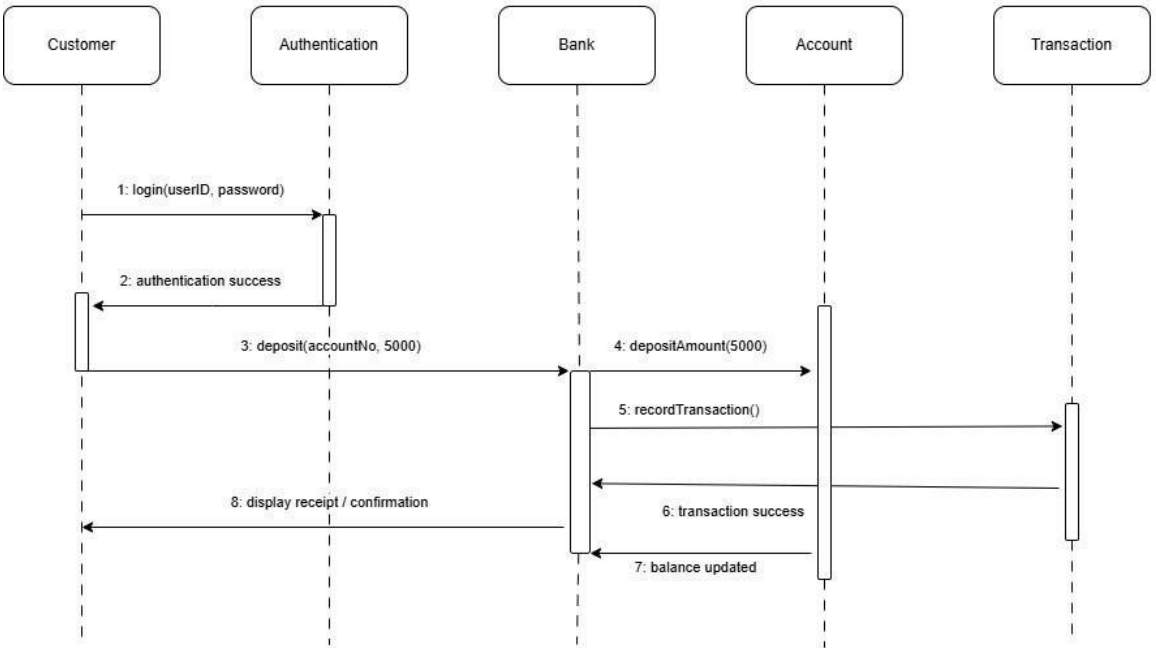


4. Sequence diagram (use UML notations)

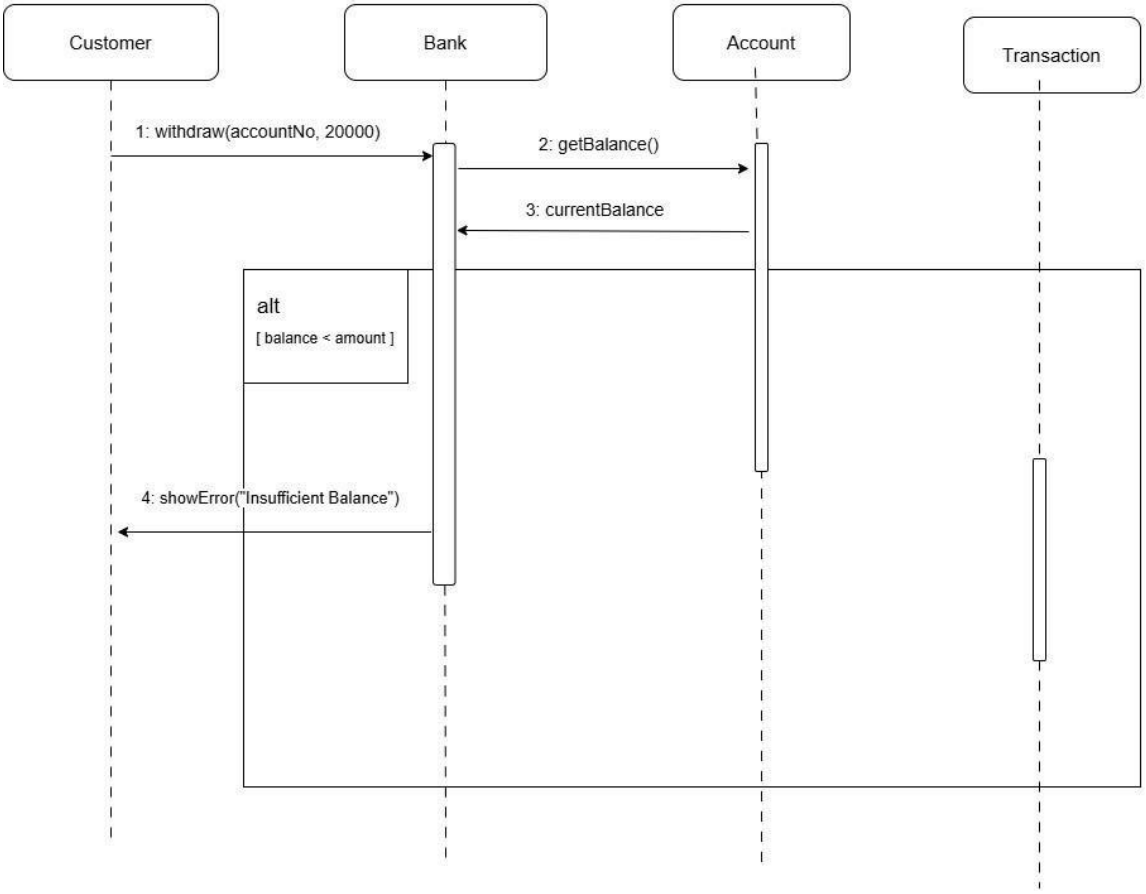
- Choose key use cases and draw their sequence diagram. First describe the scenario in few sentences. Then draw the sequence diagram.

SCENARIO : A customer logs into the Bank Management System using valid credentials. After successful authentication, the customer deposits ₹5,000 into the account. The system processes the deposit, updates the account balance, records the transaction, and displays a confirmation message to the customer.

Sequence diagram - 1 : Customer Login and Deposit Money



- Choose key use cases and draw their sequence diagram. First describe the scenario in few sentences. Then draw the sequence diagram.

SCENARIO : A customer attempts to withdraw ₹20,000 from the account. The system checks the available balance by retrieving account details. Since the balance is insufficient, the system rejects the transaction and displays an error message indicating insufficient balance.

Sequence Diagram 2: Withdraw Money with Insufficient Balance